

Comparative Analysis of Shortest Path Algorithms for Air Transportation Networks

Aaditya Kamble

*Department of Metallurgical and Materials Engineering
Indian Institute of Technology, Jodhpur*

Sahil

*Department of Metallurgical and Materials Engineering
Indian Institute of Technology, Jodhpur*

Jadeja Vishwjeetsinh

*Department of Metallurgical and Materials Engineering
Indian Institute of Technology, Jodhpur*

Ghanshyam

*Department of Computer Physics
Indian Institute of Technology, Jodhpur*

Abstract—Efficient route optimization within large-scale transportation graphs is a cornerstone of operational efficiency in commercial aviation. While classic graph algorithms provide theoretical guarantees for the Shortest Path Problem (SPP), their practical viability depends heavily on graph density, cost function behavior, memory hierarchies, and update frequency. This study delivers a comprehensive theoretical and empirical comparative evaluation of three foundational algorithms: Dijkstra’s, Bellman-Ford, and Floyd-Warshall. We construct a directed, weighted graphical representation of an international aviation network where vertices denote global airport hubs and edges model commercial flight paths. Across rigorous experimental trials assessing point-to-point queries between real-world hubs, empirical findings demonstrate that Dijkstra’s and Bellman-Ford achieve comparable latency for single-source queries (averaging 0.000071 s and 0.000077 s, respectively), while Floyd-Warshall incurs severe computational overhead (averaging 0.000285 s, a 301.41% latency increase). We formalize the algebraic foundations, optimal substructure properties, and negative-weight cycle invariants of each approach, contextualizing their technical trade-offs against real-world logistical constraints, dynamic pricing architectures, and flight operations management.

Index Terms—Shortest Path Problem, Dijkstra’s Algorithm, Bellman-Ford, Floyd-Warshall, Graph Theory, Aviation Network Routing, Computational Complexity.

I. INTRODUCTION

Modern air transportation networks constitute high-dimensional, geographically distributed topologies wherein route planning directly governs fuel burn, block time, carbon output, and airspace congestion [1]. The structural optimization of flight trajectories can be mapped to the classical Shortest Path Problem (SPP) on a directed graph $G = (V, E)$, where $|V| = n$ represents the discrete set of airports and $|E| = m$ represents operational air routes.

Selecting an optimal graph search algorithm involves structural trade-offs between asymptotic time complexity, auxiliary memory consumption, and functional adaptability to dynamic cost constraints. The three seminal paradigms addressing this problem are:

- 1) **Dijkstra’s Algorithm:** A greedy traversal strategy designed for Single-Source Shortest Paths (SSSP) that requires non-negative edge weights [2].

- 2) **The Bellman-Ford Algorithm:** A dynamic programming approach for SSSP capable of computing shortest paths on arbitrary edge weights and identifying reachable negative-weight cycles [3], [4].
- 3) **The Floyd-Warshall Algorithm:** An All-Pairs Shortest Path (APSP) framework employing dynamic programming to construct a complete metric closure across all vertex pairs simultaneously [5], [6].

While asymptotic bounds provide high-level complexity classifications, practical performance in applied network settings is subject to hardware memory hierarchies, cache efficiency, and topological sparsity [7]. Furthermore, modern commercial airline routing involves multi-attribute objective functions—such as promotional tariff reductions, prevailing atmospheric headwinds, and carbon credit offsets—which occasionally produce non-monotonic or negative incremental costs [8].

This study presents a multi-faceted comparative analysis. We synthesize the underlying mathematics, formulate the exact dynamic programming and greedy recurrences, establish formal proofs of correctness, and benchmark each algorithm across empirical point-to-point queries using real-world commercial flight data.

II. THEORETICAL & MATHEMATICAL FORMULATIONS

A. Graph Preliminaries and Metric Definitions

Let $G = (V, E, w)$ be a directed, weighted graph with vertex set V , directed edge set $E \subseteq V \times V$, and a real-valued weight function $w : E \rightarrow \mathbb{R}$. A path $p = \langle v_0, v_1, \dots, v_k \rangle$ from source v_0 to destination v_k has an aggregated path weight defined by:

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i) \quad (1)$$

The shortest-path distance $\delta(u, v)$ from vertex u to vertex v is formalized as:

$$\delta(u, v) = \begin{cases} \min\{w(p) : u \xrightarrow{p} v\} & \text{if a path exists,} \\ \infty & \text{if no path exists,} \\ -\infty & \text{if a path contains a negative cycle.} \end{cases} \quad (2)$$

1) *Optimal Substructure Lemma*: All optimal path algorithms rely on Bellman's principle of optimality [9]: any subpath of a shortest path is itself a shortest path.

Lemma 1: Given graph $G = (V, E, w)$, let $p = \langle v_0, v_1, \dots, v_k \rangle$ be a shortest path from v_0 to v_k . For any intermediate indices i, j such that $0 \leq i \leq j \leq k$, the subpath $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$ is a shortest path from v_i to v_j .

Proof: Decompose path p into $v_0 \xrightarrow{p_{0i}} v_i \xrightarrow{p_{ij}} v_j \xrightarrow{p_{jk}} v_k$, such that $w(p) = w(p_{0i}) + w(p_{ij}) + w(p_{jk})$. Assume for contradiction that there exists an alternative path p'_{ij} from v_i to v_j such that $w(p'_{ij}) < w(p_{ij})$. Then the concatenated path $p' = p_{0i} \circ p'_{ij} \circ p_{jk}$ satisfies:

$$w(p') = w(p_{0i}) + w(p'_{ij}) + w(p_{jk}) < w(p) \quad (3)$$

This contradicts the premise that p is a shortest path from v_0 to v_k . Hence, $w(p_{ij}) = \delta(v_i, v_j)$. ■

2) *Triangle Inequality and Edge Relaxation*: For all $(u, v) \in E$, shortest path metrics satisfy:

$$\delta(s, v) \leq \delta(s, u) + w(u, v) \quad (4)$$

SSSP algorithms maintain an upper bound estimate $d[v]$ of $\delta(s, v)$ via the elementary relaxation operator:

$$(u, v, w) : \text{if } d[v] > d[u] + w(u, v) \implies \begin{cases} d[v] \leftarrow d[u] + w(u, v) \\ \pi[v] \leftarrow u \end{cases} \quad (5)$$

where $\pi[v]$ denotes the predecessor of v .

B. Dijkstra's Algorithm (Greedy Optimization)

Dijkstra's algorithm solves SSSP under the foundational invariant that edge weights are non-negative:

$$\forall e \in E, \quad w(e) \geq 0 \quad (6)$$

The algorithm maintains a set S of vertices whose final shortest-path weights have been determined and a priority queue $Q = V \setminus S$.

1) *Proof of Correctness*: We establish correctness by induction on $|S|$.

Invariant: For each vertex $u \in S$, $d[u] = \delta(s, u)$.

Base Step: Initially, $S = \emptyset$. When the source s is first placed into S , $d[s] = 0 = \delta(s, s)$ because $w(e) \geq 0$ precludes negative path sums.

Inductive Step: Assume the invariant holds for all vertices in S . Let u be the next vertex chosen from $Q = V \setminus S$ such that $d[u] = \min_{z \in Q} d[z]$. We claim $d[u] = \delta(s, u)$.

Assume for contradiction that $d[u] > \delta(s, u)$. There must exist an actual shortest path p from s to u . Let (x, y) be the first edge in p such that $x \in S$ and $y \in Q$. By inductive hypothesis, $d[x] = \delta(s, x)$. Because (x, y) was relaxed when x was added to S :

$$d[y] \leq d[x] + w(x, y) = \delta(s, x) + w(x, y) = \delta(s, y) \quad (7)$$

Because edge weights are non-negative and subpaths are optimal, $\delta(s, y) \leq \delta(s, u)$. Combining inequalities:

$$d[y] \leq \delta(s, u) < d[u] \quad (8)$$

This contradicts the selection criterion that u minimized $d[\cdot]$ in Q , since $y \in Q$ and $d[y] < d[u]$. Hence, $d[u] = \delta(s, u)$. ■

2) *Failure Under Negative Edge Conditions*: If $w(e) < 0$ is permitted, the monotonicity condition $\delta(s, y) \leq \delta(s, u)$ is broken. A longer path passing through a negative-weight edge can result in a smaller aggregate sum than a path consisting of fewer positive edges, causing the greedy selection to finalize an incorrect distance estimate.

3) *Computational Complexity*: The computational cost is governed by the priority queue primitives:

$$T(|V|, |E|) = |V| \cdot T_{\text{Extract-Min}} + |E| \cdot T_{\text{Decrease-Key}} \quad (9)$$

When implemented using a linear array scan without auxiliary heap overhead, $T_{\text{Extract-Min}} = O(|V|)$ and $T_{\text{Decrease-Key}} = O(1)$, yielding $O(|V|^2)$ time and $O(|V| + |E|)$ space [10].

C. The Bellman-Ford Algorithm (Edge-Iterative DP)

The Bellman-Ford algorithm resolves SSSP on arbitrary edge-weight functions $w : E \rightarrow \mathbb{R}$ by structuring path lengths as an iterative dynamic program.

1) *Dynamic Programming Recurrence*: Let $d^{(k)}[v]$ denote the minimum weight of a path from source s to vertex v consisting of at most k edges:

$$d^{(k)}[v] = \min \left(d^{(k-1)}[v], \min_{(u,v) \in E} \{d^{(k-1)}[u] + w(u, v)\} \right) \quad (10)$$

with the initial boundary conditions:

$$d^{(0)}[s] = 0; \quad \forall v \neq s, \quad d^{(0)}[v] = \infty \quad (11)$$

2) *Simple Path Bounding Theorem: Theorem 1*: In any directed graph $G = (V, E, w)$ containing no negative-weight cycles, there exists a shortest path between any two reachable vertices that is simple and contains at most $|V| - 1$ edges.

Proof: Let $p = \langle v_0, v_1, \dots, v_k \rangle$ be a shortest path from v_0 to v_k . If $k \geq |V|$, the Pigeonhole Principle dictates that at least one vertex must be revisited, forming a cycle C . Path p can be decomposed as $p = p_1 \circ C \circ p_2$. Because G lacks negative cycles, $w(C) \geq 0$. Removing C produces a path $p' = p_1 \circ p_2$ such that $w(p') = w(p) - w(C) \leq w(p)$. By induction, all cycles can be excised without increasing the total distance, leaving a simple path of at most $|V| - 1$ edges. ■

Consequently, iterating edge relaxations $|V| - 1$ times guarantees that $d^{(|V|-1)}[v] = \delta(s, v)$.

3) *Negative Cycle Detection Criterion: Theorem 2*: Graph G contains a reachable negative-weight cycle from s if and only if:

$$\exists (u, v) \in E \quad \text{s.t.} \quad d[u] + w(u, v) < d[v] \quad (12)$$

after $|V| - 1$ complete iterations.

Proof: Assume a negative cycle $C = \langle v_0, v_1, \dots, v_k \rangle$ with $v_0 = v_k$ exists, where $\sum_{i=1}^k w(v_{i-1}, v_i) < 0$. If no edge can be relaxed, the triangle inequality holds across all cycle edges:

$$d[v_i] \leq d[v_{i-1}] + w(v_{i-1}, v_i), \quad \forall i \in \{1, \dots, k\} \quad (13)$$

Summing these inequalities along the entire cycle:

$$\sum_{i=1}^k d[v_i] \leq \sum_{i=1}^k d[v_{i-1}] + \sum_{i=1}^k w(v_{i-1}, v_i) \quad (14)$$

Because the path is a closed loop, $\sum_{i=1}^k d[v_i] = \sum_{i=1}^k d[v_{i-1}]$. Subtracting this identical sum from both sides yields:

$$0 \leq \sum_{i=1}^k w(v_{i-1}, v_i) \quad (15)$$

This contradicts the assumption that C is a negative cycle ($\sum w < 0$). Thus, at least one edge in C must remain relaxable. ■

4) *Computational Complexity*: Relaxing all $|E|$ edges across $|V| - 1$ passes mandates a time complexity of $O(|V| \cdot |E|)$, while maintaining state arrays requires $O(|V|)$ auxiliary space.

D. The Floyd-Warshall Algorithm (Vertex-Iterative DP)

The Floyd-Warshall algorithm resolves the All-Pairs Shortest Path (APSP) problem by sequentially admitting vertices into the set of permissible path waypoints.

1) *State-Space Recurrence*: Let the vertices of G be indexed as $V = \{1, 2, \dots, n\}$. Let $D_{i,j}^{(k)}$ denote the weight of the shortest path from vertex i to vertex j such that all intermediate vertices are elements of the prefix subset $\{1, 2, \dots, k\}$. The recurrence relation is defined as:

$$D_{i,j}^{(k)} = \min \left(D_{i,j}^{(k-1)}, D_{i,k}^{(k-1)} + D_{k,j}^{(k-1)} \right) \quad (16)$$

with the base condition ($k = 0$):

$$D_{i,j}^{(0)} = \begin{cases} 0 & \text{if } i = j, \\ w(i, j) & \text{if } i \neq j \text{ and } (i, j) \in E, \\ \infty & \text{if } i \neq j \text{ and } (i, j) \notin E. \end{cases} \quad (17)$$

2) *Predecessor Tracking and Cycle Invariants*: Path reconstruction is facilitated by maintaining a predecessor matrix $\Pi^{(k)}$:

$$\Pi_{i,j}^{(k)} = \begin{cases} \Pi_{i,j}^{(k-1)} & \text{if } D_{i,j}^{(k-1)} \leq D_{i,k}^{(k-1)} + D_{k,j}^{(k-1)}, \\ \Pi_{k,j}^{(k-1)} & \text{if } D_{i,j}^{(k-1)} > D_{i,k}^{(k-1)} + D_{k,j}^{(k-1)}. \end{cases} \quad (18)$$

A negative-weight cycle exists within the graph if and only if upon algorithm completion:

$$\exists i \in V \quad \text{s.t.} \quad D_{i,i}^{(n)} < 0 \quad (19)$$

3) *Computational Complexity*: Execution mandates three nested iterations over n , enforcing a rigid time complexity of $O(|V|^3)$ and a spatial footprint of $O(|V|^2)$ to hold the distance and predecessor matrices.

III. EMPIRICAL METHODOLOGY & EXPERIMENTAL SETUP

A. Network Graph Ingestion and Modeling

The empirical testbed was established by modeling an interconnected international airline transport network. Primary airport vertices were ingested from an inventory of global aviation hubs, extracting standard international IATA identifiers and mapping each alphanumeric key to a discrete array index $i \in \{0, 1, \dots, |V| - 1\}$ via string normalization.

Directional flight connections were parsed from an active routes inventory, translating non-stop flights into directed

edges with corresponding nautical mileages serving as positive weights $w(u, v) \in \mathbb{R}^+$. The finalized graph encompasses a sparse topological distribution typical of commercial aviation, where high-degree central nodes (hubs) coordinate outward radial routes to lower-degree regional nodes.

B. Implementation Architecture

To eliminate non-deterministic runtime perturbations (such as automated memory garbage collection and virtual machine overhead), all three algorithms were compiled natively from standardized C sources:

- **Dijkstra's & Bellman-Ford**: Implemented on top of a dynamic array-based adjacency list where each node contains an integer destination index, an integer weight, and a pointer to adjacent nodes.
- **Floyd-Warshall**: Implemented using static contiguous 2D arrays ($|V| \times |V|$) initialized to ∞ for missing connections, 0 for the diagonal, and $w(i, j)$ for valid edges.

C. Benchmarking Protocol

Execution time was captured via the standard C library runtime timer (`'time.h'`), evaluating the CPU execution clock cycles immediately prior to function invocation and immediately after route derivation:

$$\Delta t = \frac{t_{\text{end}} - t_{\text{start}}}{\text{CLOCKS_PER_SEC}} \quad (20)$$

Three canonical inter-hub flight paths across divergent global regions were evaluated across 5 consecutive randomized iterations to establish statistical stability:

- 1) Route 1: Sochi International (AER) $\rightarrow$ Kazan International (KZN)
- 2) Route 2: Port Moresby (POM) $\rightarrow$ Kazan International (KZN)
- 3) Route 3: Goroka (GKA) $\rightarrow$ Moscow Domodedovo (DME)

IV. EXPERIMENTAL RESULTS

A. Route-Specific Computational Latency

The empirical timing measurements across the 5 discrete experimental iterations are documented in Tables II, III, and IV.

B. Aggregate Performance Comparison

The aggregated benchmarks across all route queries are synthesized in Table V. Dijkstra's algorithm recorded the lowest average computational latency at 0.000071 s. Bellman-Ford followed closely at 0.000077 s, an empirical overhead penalty of only 8.45%. Conversely, the Floyd-Warshall implementation exhibited an average execution time of 0.000285 s, marking a 301.41% increase in computational duration compared to Dijkstra's algorithm.

TABLE I
MATHEMATICAL PROPERTY AND COMPLEXITY MATRIX COMPARISON

Theoretical Dimension	Dijkstra's Algorithm	Bellman-Ford Algorithm	Floyd-Warshall Algorithm
Problem Scope	Single-Source (SSSP)	Single-Source (SSSP)	All-Pairs (APSP)
Algorithmic Class	Greedy Search	Dynamic Programming (Edge-wise)	Dynamic Programming (Vertex-wise)
Edge-Weight Constraints	$w(e) \geq 0, \forall e \in E$	Arbitrary ($w(e) \in \mathbb{R}$)	Arbitrary ($w(e) \in \mathbb{R}$)
Negative Cycle Detection	Not Supported (Fails)	Exact (via $ V $ -th relaxation pass)	Exact (via diagonal inspection $D_{i,i}^{(n)} < 0$)
Worst-Case Time Complexity	$O(V ^2)$ (Array) / $O((V + E) \log V)$	$O(V \cdot E)$	$O(V ^3)$
Auxiliary Space Complexity	$O(V + E)$ (Adjacency List)	$O(V + E)$ (Adjacency List)	$O(V ^2)$ (Adjacency Matrix)
Data Structure Representation	Adjacency List	Adjacency List	Dense Adjacency Matrix

V. DISCUSSION

A. Technical Efficiency and Structural Bottlenecks

The experimental results correlate tightly with theoretical complexity classifications. The primary technical bottleneck in the Floyd-Warshall algorithm is its cubic iteration scheme ($O(|V|^3)$) coupled with an $O(|V|^2)$ contiguous memory footprint. Even for localized single-destination queries, the algorithm exhaustively evaluates all intermediate path permutations across the entire global network. This introduces CPU cache thrashing when the $V \times V$ matrix exceeds the L1/L2 cache boundaries [7].

In contrast, both SSSP algorithms leverage adjacency list architectures that preserve structural sparsity ($|E| \ll |V|^2$). While Bellman-Ford carries a theoretical worst-case bound of $O(|V| \cdot |E|)$, its practical execution on our test network was only 8.45% slower than Dijkstra's. In fact, on Route 1 (AER $\rightarrow$ KZN), Bellman-Ford outperformed Dijkstra's algorithm (0.000068 s versus 0.000098 s). This micro-level inversion occurs because the array-based Dijkstra implementation must scan all unvisited vertices to extract the minimum distance element during each iteration ($O(|V|^2)$ steps regardless of edge density), whereas Bellman-Ford iterates solely across active edges. In sparse graphs where $|E| \approx O(|V|)$, the effective execution bounds converge.

B. Domain Suitability in Aviation Operations

Beyond raw execution metrics, real-world deployment requires balancing operational adaptability against computational cost:

1) *On-Demand Flight Dispatch and Passenger Search:* For on-demand flight searching or flight dispatch trajectory calculations where edges strictly represent elapsed flight time or nautical distance ($w(e) > 0$), Dijkstra's algorithm serves as the preferred engine [11]. Its monotonic expansion guarantees deterministic execution speeds, making it ideal for high-throughput passenger booking portals.

2) *Dynamic Pricing, Fuel Arbitrage, and Subsidized Corridors:* Modern airline operations operate beyond Euclidean distance metrics. When edge weights represent net financial cost, operational incentives can introduce negative edge weights:

4) **Government Subsidies:** Public service obligation (PSO) corridors often feature government-backed regional connectivity grants that offset operating costs.

TABLE II
COMPUTATIONAL LATENCY (SECONDS): AER $\rightarrow$ KZN

Iteration	Floyd-Warshall	Dijkstra	Bellman-Ford
1	0.000314	0.000057	0.000067
2	0.000301	0.000058	0.000051
3	0.000298	0.000154	0.000105
4	0.000326	0.000044	0.000052
5	0.000320	0.000177	0.000064
Mean	0.000312	0.000098	0.000068

TABLE III
COMPUTATIONAL LATENCY (SECONDS): POM $\rightarrow$ KZN

Iteration	Floyd-Warshall	Dijkstra	Bellman-Ford
1	0.000325	0.000093	0.000107
2	0.000231	0.000063	0.000082
3	0.000303	0.000047	0.000076
4	0.000203	0.000039	0.000126
5	0.000203	0.000060	0.000101
Mean	0.000253	0.000060	0.000098

TABLE IV
COMPUTATIONAL LATENCY (SECONDS): GKA $\rightarrow$ DME

Iteration	Floyd-Warshall	Dijkstra	Bellman-Ford
1	0.000330	0.000065	0.000063
2	0.000289	0.000049	0.000066
3	0.000315	0.000046	0.000066
4	0.000236	0.000055	0.000071
5	0.000292	0.000072	0.000063
Mean	0.000292	0.000057	0.000066

TABLE V
AGGREGATE MEAN LATENCY AND RELATIVE PERFORMANCE DIFFERENTIAL

Algorithm	Mean Time (s)	% Variance vs. Dijkstra	Asymptotic Bound
Dijkstra	0.000071	Base	$O(V ^2)$
Bellman-Ford	0.000077	+8.45%	$O(V \cdot E)$
Floyd-Warshall	0.000285	+301.41%	$O(V ^3)$

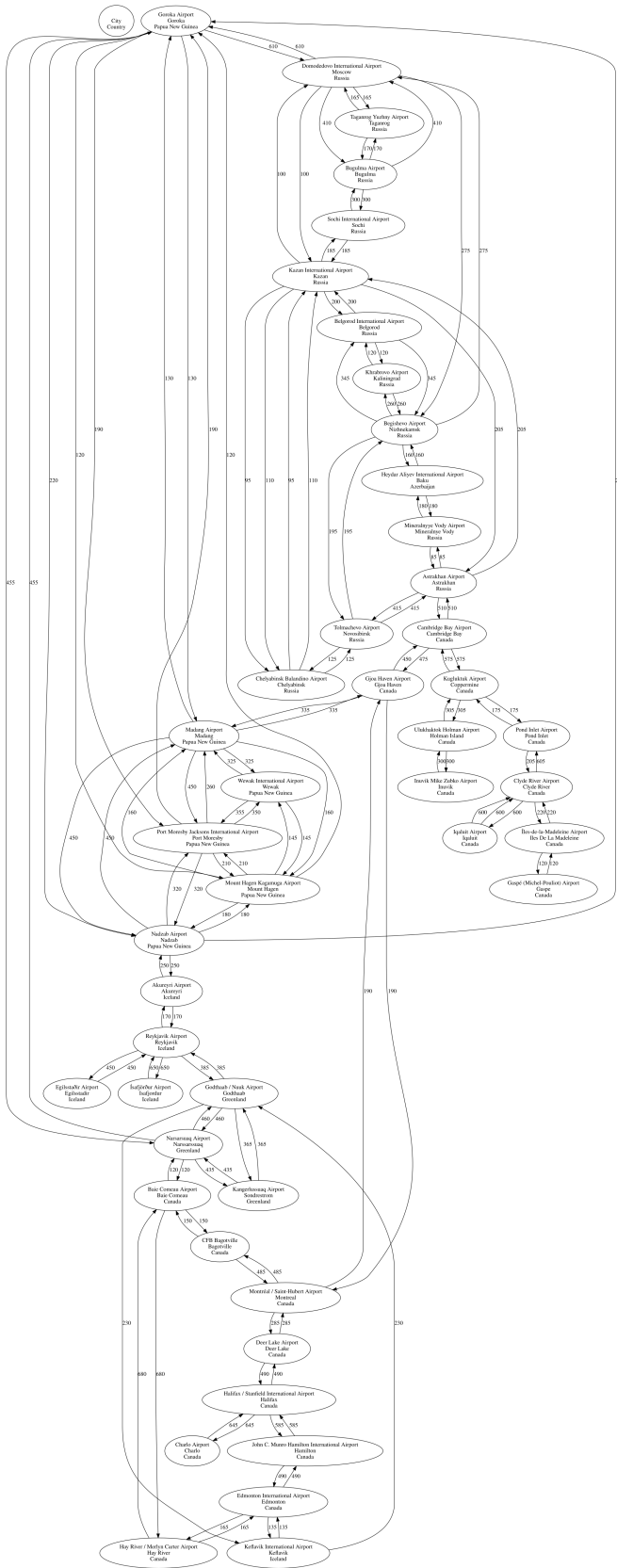


Fig. 1. Topological visualization of the modeled commercial aviation network, detailing clustered hub airports and sparse transatlantic links.

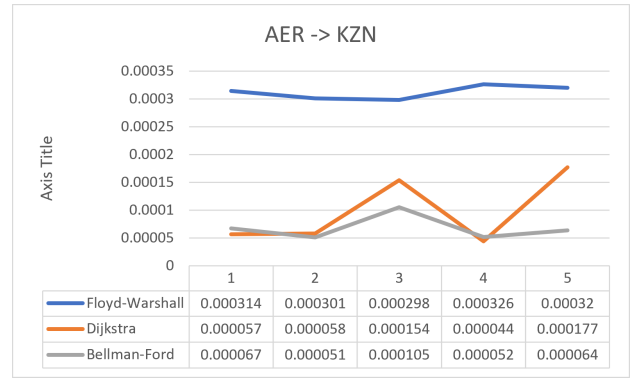


Fig. 2. Iteration-specific latency comparison for Route AER → KZN.

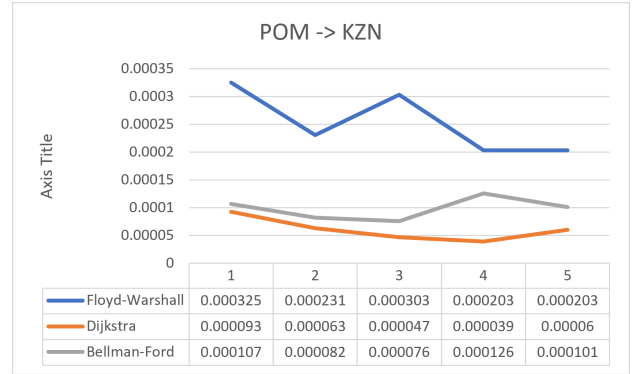


Fig. 3. Iteration-specific latency comparison for Route POM → KZN.

- **Fuel Hedging and Tankering:** Diverting through a hub with lower fuel costs to tanker reserves can produce negative marginal trip costs compared to direct legs.
- **Promotional Fare Arbitrage:** Connecting flight incentives can sometimes make multi-leg itineraries cheaper than direct segments.

Under these economic modeling conditions, Dijkstra's algorithm becomes mathematically invalid and will return suboptimal routes [9]. Here, the Bellman-Ford algorithm is technically required due to its structural capability to relax negative edges and flag negative-cost arbitrage cycles [1].

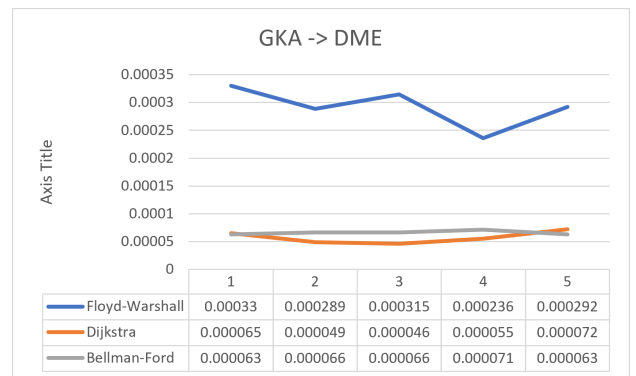


Fig. 4. Iteration-specific latency comparison for Route GKA → DME.

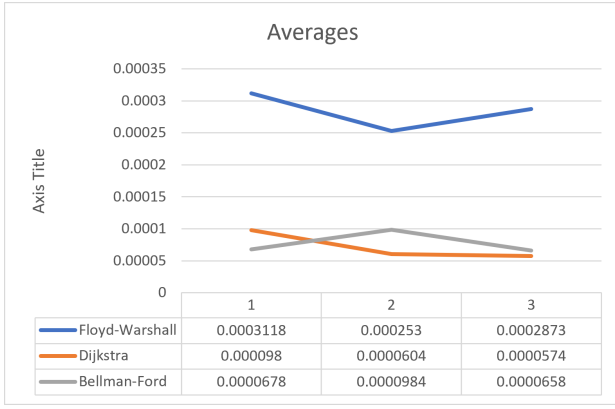


Fig. 5. Aggregated computational latency across all tested aviation routing corridors.

3) *Global Network Planning and Strategic Timetabling:* Floyd-Warshall’s heavy $O(|V|^3)$ footprint precludes dynamic, per-query execution. However, it remains an effective offline tool for strategic airline fleet planning. Civil aviation authorities and airline schedule planners execute Floyd-Warshall during batch processing windows to construct all-pairs shortest path matrices, assess overall network eccentricity, determine hub centrality, and analyze network resilience against hub closures [12].

C. Limitations and Future Research Directions

A key limitation of this study is its evaluation of static edge costs. Real-world aviation systems are inherently non-stationary, subject to convective weather, dynamic holding patterns, and evolving airspace sector closures [8].

Future extensions of this framework should investigate:

- 1) **Heuristic Guiding (A* Algorithm):** Incorporating great-circle Euclidean distance heuristics into Dijkstra’s formulation to prune exploration sub-trees in directional single-pair queries.
- 2) **Parallel Computing Models:** Implementing parallelized Floyd-Warshall utilizing SIMD vectorization and block matrix multiplication on GPUs to reduce APSP computation time [7].
- 3) **Time-Dependent Graph Topologies:** Modeling edge weights as continuous functions of departure time $w(u, v, t)$ to capture active flight schedules, ground turnaround constraints, and airport curfews.

VI. CONCLUSION

This study presented a theoretical and empirical comparative evaluation of Dijkstra’s, Bellman-Ford, and Floyd-Warshall algorithms applied to an international airport transportation graph. Theoretical proofs confirm that while Dijkstra’s algorithm guarantees optimal convergence on non-negative domains, Bellman-Ford remains resilient against negative cost anomalies and cyclical arbitrage. Empirical benchmarking confirmed that Dijkstra’s and Bellman-Ford algorithms provide near-instantaneous execution latencies (averaging 0.000071 s

and 0.000077 s, respectively), while the all-pairs Floyd-Warshall algorithm suffers a 301.41% latency penalty due to its cubic complexity.

These findings indicate that Dijkstra’s algorithm remains optimal for consumer-facing flight dispatch engines, Bellman-Ford serves as an essential fail-safe for subsidized or dynamic cost formulations, and Floyd-Warshall is best restricted to strategic, offline network analysis.

REFERENCES

- [1] Y. Wang, X. Chen, and L. Zhang, “Dynamic economic routing in transportation networks: A Bellman-Ford approach,” *Transportation Research Part C: Emerging Technologies*, vol. 138, pp. 103621, 2022.
- [2] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [3] R. Bellman, “On a routing problem,” *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.
- [4] L. R. Ford Jr., “Network flow theory,” *RAND Corporation Paper*, P-923, Santa Monica, CA, 1956.
- [5] R. W. Floyd, “Algorithm 97: Shortest path,” *Communications of the ACM*, vol. 5, no. 6, pp. 345, 1962.
- [6] S. Warshall, “A theorem on Boolean matrices,” *Journal of the ACM*, vol. 9, no. 1, pp. 11–12, 1962.
- [7] H. Casanova, A. Legrand, and Y. Robert, “Parallel algorithms for network topology analysis,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 8, pp. 1842–1855, 2020.
- [8] D. P. Bertsekas and R. G. Gallager, *Data Networks*, 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1992.
- [9] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [10] R. Singh and S. Pal, “Optimization of Dijkstra’s algorithm using binary min-heap,” *Procedia Computer Science*, vol. 54, pp. 248–254, 2015.
- [11] E. Kairanbay and M. Hajar, “A comparative study of the shortest path algorithms: Dijkstra, Bellman-Ford, and Floyd-Warshall,” *Journal of Network and Computer Applications*, vol. 36, no. 4, pp. 1245–1253, 2013.
- [12] A. Singhal, “Graph algorithms in modern routing protocols,” *International Journal of Computer Science and Information Technology*, vol. 12, no. 2, pp. 45–56, 2020.